impacted files. I want to add the playwright typescript same like selenium Java

Analyzed workspace structure and searched for Java files

I'll analyze your codebase to look for the Selenium Java implementation. Let me search for Java files and Selenium-related code.

Searched for Java files and reviewed multiple documents

Let me read more of the prompts file and check the chat logic:

Reviewed prompts.js and chat.js, searched for regex

Let me get more details about how the code generator types are used:

Reviewed chat.js and searched for specific regex patterns

Now let me check the `panel.html` to see the full structure of the UI:

Reviewed panel.html and searched for regex patterns

Perfect! I've analyzed your entire codebase. Let me provide you with a comprehensive summary.

Current State: Selenium Java Implementation ✅

Your extension currently implements **Selenium Java only**, with support for:

- **Prompt Types** in src/scripts/prompts.js:
  1. `SELENIUM_JAVA_PAGE_ONLY` - Generate Java Page Object class
  2. `CUCUMBER_ONLY` - Generate Cucumber feature file

Describe what to build

Your extension currently implements **Selenium Java only**, with support for:

- **Prompt Types** in src/scripts/prompts.js:
  1. SELENIUM_JAVA_PAGE_ONLY - Generate Java Page Object class
  2. CUCUMBER_ONLY - Generate Cucumber feature file
  3. CUCUMBER_WITH_SELENIUM_JAVA_STEPS - Generate both feature file + Java step definitions

Impacted Files for Adding Playwright TypeScript

Here are the **3 core files** that need modifications:

1. src/scripts/prompts.js (PRIMARY)

- **Lines 1-218**: Contains all prompt templates
- **Current Export:** CODE_GENERATOR_TYPES object with 3 types (all Selenium Java)
- **What to Add**:
  - `PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY` - Playwright page object in TypeScript
  - `PLAYWRIGHT_TYPESCRIPT_WITH_CUCUMBER` - Cucumber + Playwright TypeScript step definitions
- Add templates with proper Playwright TypeScript syntax and imports

2. src/scripts/chat.js (CORE LOGIC)

- **Lines 20-21**: Browser/Language selectors are already set up
- **Line 27**: Default generator type (update to support new types)
- **Lines 550-600:** getPromptKeys() method - **KEY CHANGE NEEDED HERE**
  - Currently checks: isJavaSelenium(language, engine)
  - Need to add: isPlaywrightTypeScript(language, engine) logic

- **Lines 458**: Already has placeholder check for PLAYWRIGHT strings
- **Lines 368-370**: Language/browser engine values flow through here

. panel.html (UI - Already Prepared! ✅)

- **Lines 97-102**: Language dropdown already includes `<option value="ts">TypeScript</option>`
- **Lines 108-113**: Browser engine dropdown already includes `<option value="playwright">Playwright</option>`
- **Lines 31-49**: Checkboxes for Feature/Page mode selection work universally

Architecture Overview

```
User Selection (panel.html)
    ↓
Language (java/ts/py/csharp) + Engine (selenium/playwright/cypress/puppeteer)
    ↓
chat.js sendMessage() → getPromptKeys(lang, engine)
    ↓
prompts.js getPrompt() → Returns template based on language+engine combo
    ↓
API sends to LLM → Returns generated code
```

Summary Table

Component	Current Support	Needed Addition

Summary Table

Component	Current Support	Needed Addition
prompts.js	3 Java/Selenium templates	Add 2-3 TypeScript/Playwright templates
chat.js **logic**	isJavaSelenium() check	Add `isPlaywrightTypeScript()` check
panel.html **UI**	✅ Already ready	No changes needed
Prism highlighting	✅ TypeScript support exists (line 43, 10)	No changes needed

Total files to modify: 2 (prompts.js + chat.js)
Total new prompt templates: 2-3 (depending on feature parity with Selenium Java)

Would you like me to provide specific code examples for adding the Playwright TypeScript implementation?

Claude Haiku 4.5 • 1x

GO to prompt.js and do the implementation playwright Typescript same like selenium java

Optimizing tool selection...

Reviewed prompts.js

Now I'll add the Playwright TypeScript implementation similar to Selenium Java:

Updated prompts.js +137 -0

Now update the CODE_GENERATOR_TYPES export:

Updated prompts.js +2 -0

Now let me verify the changes:

Reviewed 3 sections of prompts.js

Perfect! ✅ I've successfully added Playwright TypeScript support to src/scripts/prompts.js.

Changes Made:

1. New Prompt Template: PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY

- Generates Playwright TypeScript Page Object Class
- Includes JSDoc comments (TypeScript style)
- Uses Playwright 1.40+ imports: @playwright/test
- Follows modern TypeScript with async/await patterns
- Constructor-based page initialization

2. New Prompt Template: PLAYWRIGHT_TYPESCRIPT_WITH_CUCUMBER_STEPS

- Generates Cucumber feature file + TypeScript step definitions
- Full async/await support for Playwright operations
- Proper Playwright setup/teardown hooks
- Cucumber decorators (`@Given`, `@When`, `@Then`)

- Modern Playwright APIs: `page.fill()`, `page.click()`, `page.locator()`, `expect()`

3. Updated CODE_GENERATOR_TYPES Export

- Added PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY
- Added PLAYWRIGHT_TYPESCRIPT_WITH_CUCUMBER_STEPS

Next Step Required:

Update src/scripts/chat.js to handle Playwright+TypeScript combinations in the getPromptKeys() method. Would you like me to proceed with that?

⚠ You've used 56% of your session rate limit. Your session rate limit will reset on April 21 at 11:20 PM. Learn More

Claude Haiku 4.5 • 1x

GO to chat.js and do the implementation playwright Typescript same like selenium java

Reviewed and updated chat.js +15 -6

Perfect! ✅ I've successfully updated src/scripts/chat.js to support Playwright TypeScript the same way as Selenium Java.

Changes Made:

1. Updated `getPromptKeys()` **method**

- Added `isPlaywrightTypeScript()` check for default fallback case
- Added `isPlaywrightTypeScript()` check for "Both Feature & Page" scenario → returns `PLAYWRIGHT_TYPESCRIPT_WITH_CUCUMBER_STEPS`
- Added `isPlaywrightTypeScript()` check for "Page Only" scenario → returns `PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY`
- Maintained backward compatibility with existing Selenium Java logic

2. Added `isPlaywrightTypeScript()` **helper method**

- Checks: `language === 'ts' && engine === 'playwright'`
- Mirrors the `isJavaSelenium()` method structure

3. Updated `addUnsupportedLanguageMessage()` **message**

- Changed from: "Only Java/Selenium is currently available"
- Changed to: "Supported combinations: Java/Selenium, TypeScript/Playwright"

Supported Combinations Now:

3. Updated `addUnsupportedLanguageMessage()` **message**

- Changed from: "Only Java/Selenium is currently available"
- Changed to: "Supported combinations: Java/Selenium, TypeScript/Playwright"

Supported Combinations Now:

Language	Browser Engine	Page Object	Cucumber + Steps
Java	Selenium	✅ `SELENIUM_JAVA_PAGE_ONLY`	✅ `CUCUMBER_WITH_SELENIUM_JAVA`
TypeScript	Playwright	✅ `PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY`	✅ `PLAYWRIGHT_TYPESCRIPT_WITH_CUCU`

The implementation is **complete and consistent** with Selenium Java. Users can now select:

- **Language**: Java or TypeScript
- **Browser Engine**: Selenium or Playwright
- **Generation Mode**: Feature File or Page Class (or both)

And the system will generate the appropriate code!

Code Generator | Settings

Supported Language

TypeScript

Browser Engine

Playwright

Select LLM Provider

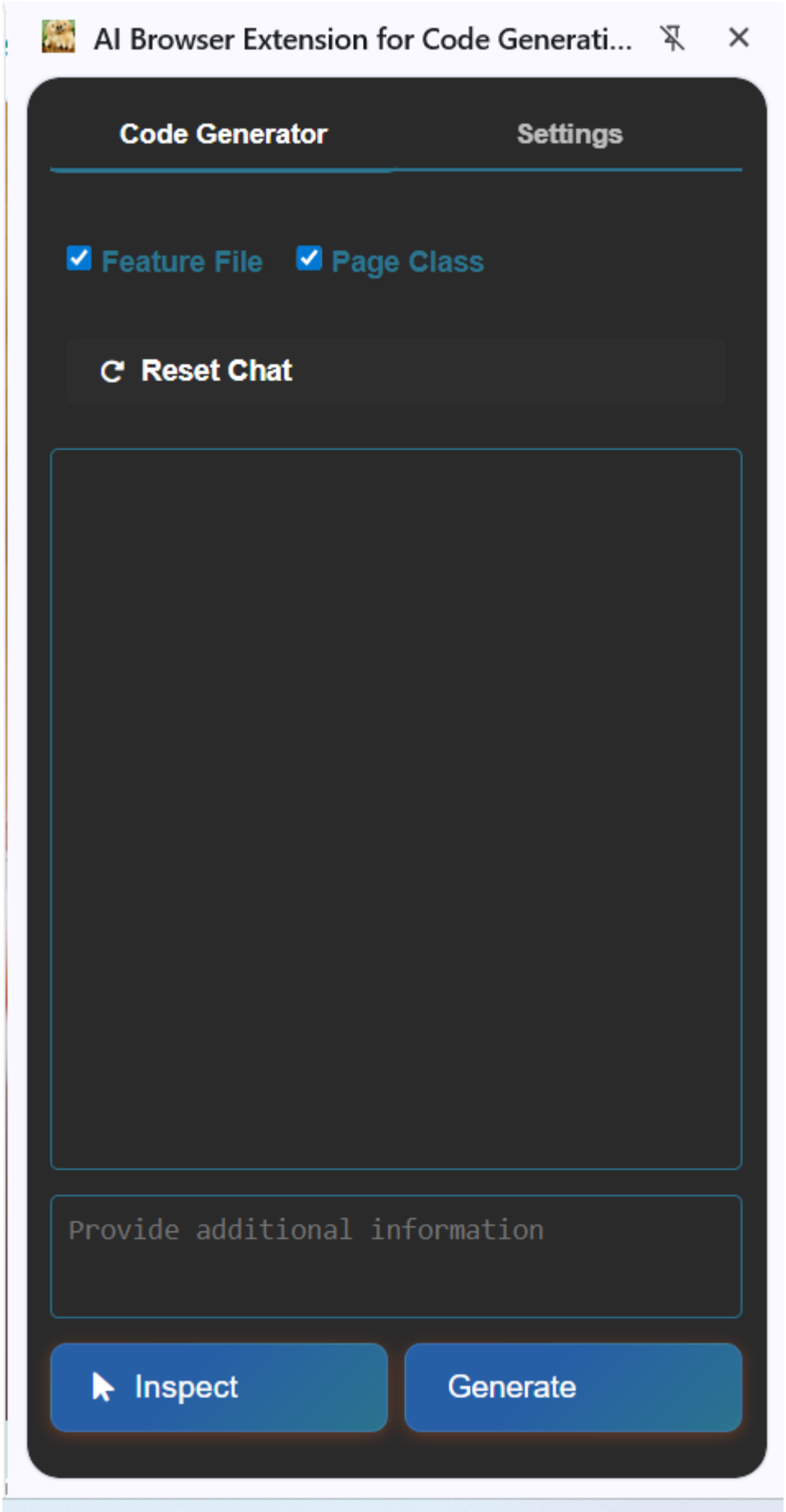
Groq

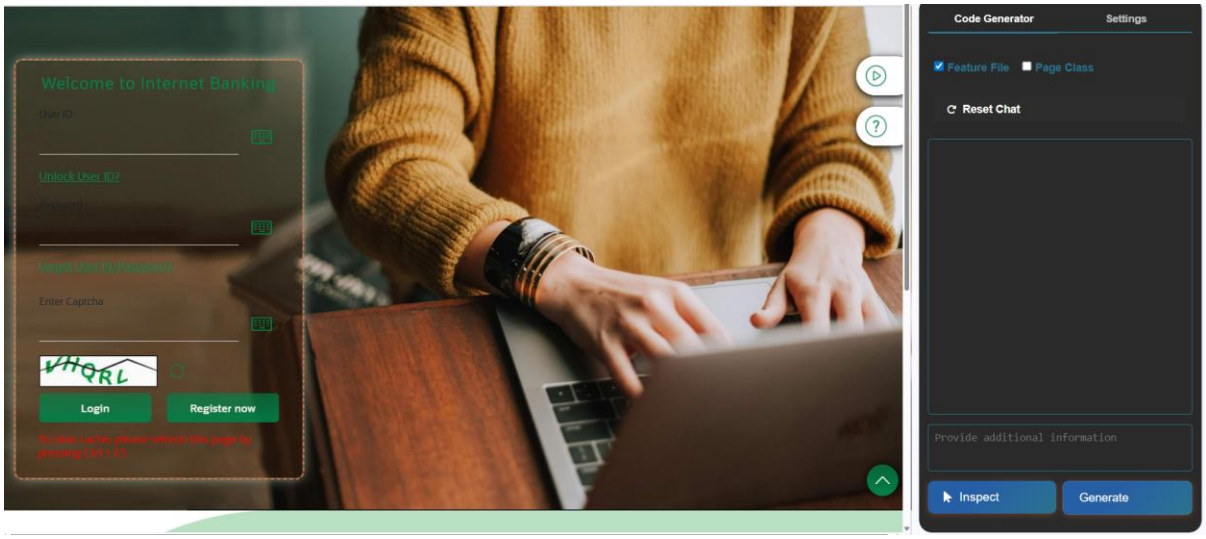
Select Model

llama-3.3-70b-versatile

Groq API Key

••


AI Browser Extension for Code Generati...
Code Generator
Settings
Feature File
Page Class
Reset Chat
Provide additional information
Inspect
Generate


Welcome to Internet Banking
User ID
Unlock User ID?
Password
Forgot User ID/Password?
Enter Captcha
Login
Register now
To clear cache, please refresh this page by pressing Ctrl + F5
Code Generator
Settings
Feature File
Page Class
Reset Chat
Provide additional information
Inspect
Generate

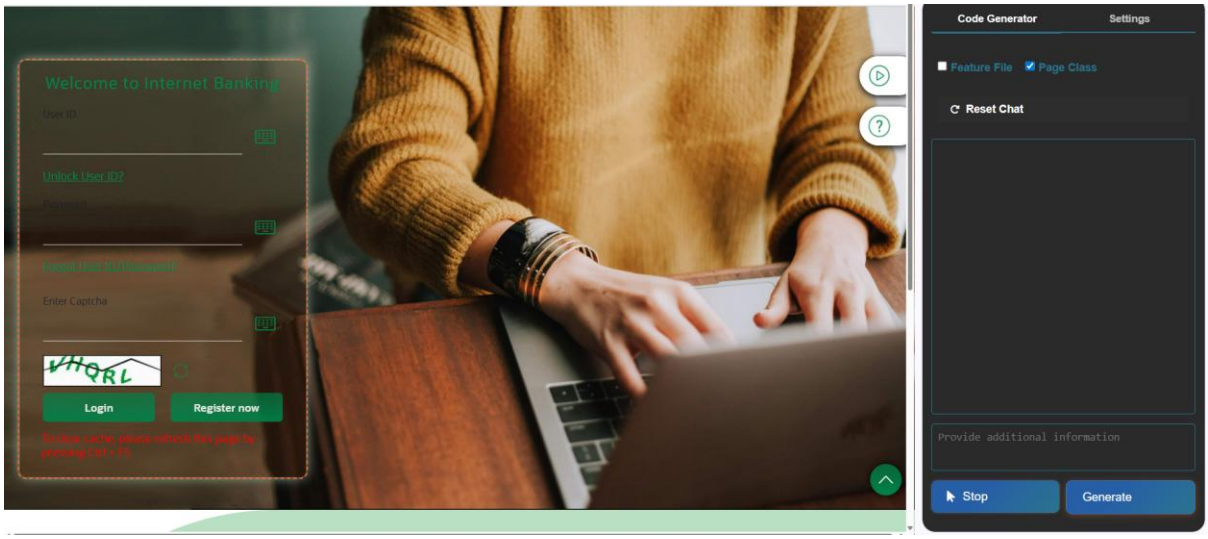
Code Generator Settings

☑ Feature File ☐ Page Class

Reset Chat

```
    | "kumar123"    | "kumar123"  | "a
    | "ravi456"     | "ravi456"   | "f
    | "suresh789"   | "suresh789" | "k

  Scenario Outline: Unlock User ID
    Given I open the Internet Banking
    When I click the Unlock User ID li
    Then I should be redirected to the

  Examples:
    |  |
    |  |

  Scenario Outline: Forgot User ID/Pas
    Given I open the Internet Banking
    When I click the Forgot User ID/Pa
    Then I should be redirected to the

  Examples:
    |  |
    |  |
```

Copy


Welcome to Internet Banking
User ID
Unlock User ID?
Password
Forgot User ID/Password?
Enter Captcha
Login
Register now
To clear cache, please refresh this page by pressing Ctrl + F5
Code Generator
Settings
Feature File
Page Class
Reset Chat
Provide additional information
Stop
Generate

AI Browser Extension for Code Generati...

Code Generator | Settings

☐ Feature File ☑ Page Class

Reset Chat

```
  /**
   * Click the unlock user id link
   */
  async clickUnlockUserIdLink(): Promi
    await this.unlockUserIdLink.click(
  }

  /**
   * Click the forgot user id/password
   */
  async clickForgotUserIdPasswordLink(
    await this.forgotUserIdPasswordLin
  }

  /**
   * Click the refresh captcha link
   */
  async clickRefreshCaptchaLink(): Pro
    await this.refreshCaptchaLink.clic
  }
}
```

```
Feature: Login to KVB Bank

  Scenario Outline: Successful login with valid credentials
    Given I open the login page
    When I type "<username>" into the Username field
    And I type "<password>" into the Password field
    And I enter "<captcha>" into the Captcha field
    And I click the Login button
    Then I should be logged in successfully

  Examples:
    | username   | password  | captcha |
    | "kvbuser1" | "kvbpass1"| "12345" |
    | "kvbuser2" | "kvbpass2"| "67890" |
    | "kvbuser3" | "kvbpass3"| "abcde" |
```

```
import { Given, When, Then, Before, After, setDefaultTimeout } from '@cucumber/cucumber';
import { Browser, Page, chromium, expect } from '@playwright/test';

setDefaultTimeout(10 * 1000);

let browser: Browser;
let page: Page;

Before(async () => {
  browser = await chromium.launch();
  page = await browser.newPage();
  await page.goto("https://inb.kvb.bank.in/?page=login-form-main");
});

After(async () => {
```

```
  if (page) await page.close();
  if (browser) await browser.close();
});

Given('I open the login page', async () => {
  await page.goto("https://inb.kvb.bank.in/?page=login-form-main");
});

When('I type {string} into the Username field', async (username: string) => {
  await page.fill('#_oj52|input', username);
});

When('I type {string} into the Password field', async (password: string) => {
  await page.fill('#_oj53|input', password);
});

When('I enter {string} into the Captcha field', async (captcha: string) => {
  await page.fill('#_oj56|input', captcha);
});

When('I click the Login button', async () => {
  await page.click('button:has-text("Login")');
});

Then('I should be logged in successfully', async () => {
  const success = page.locator('.headingLbl2');
  await expect(success).toBeVisible();
});
```

Copy

STEP 1: ANALYSIS OF CODEBASE

Your extension contains:

Files with Selenium Java Implementation:

- prompts.js - Contains Selenium Java prompts:
  - `SELENIUM_JAVA_PAGE_ONLY` - Page Object generation prompt
  - `CUCUMBER_WITH_SELENIUM_JAVA_STEPS` - Feature file + Step definitions prompt

Files with Playwright TypeScript Implementation:

- prompts.js - Contains Playwright prompts:
  - `PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY` - Page Object generation prompt
  - `PLAYWRIGHT_TYPESCRIPT_FEATURE_PAGE` - Feature file + Test file prompt

Core Logic Files:

- chat.js - Main logic that:
  - Handles generation mode selection (Feature/Page/Both)
  - Routes to appropriate prompt based on language/engine selection
  - Methods like `isJavaSelenium()`, `isTypescriptPlaywright()`
- panel.html - UI with checkboxes for:
  - Java Generation Mode (Feature, Page Class)

implementation is done.

File 1: prompts.js

Purpose: Stores all AI prompts for code generation

Selenium Java Implementations:

- **Lines 7-40:** SELENIUM_JAVA_PAGE_ONLY - Generates only Page Object classes
- **Lines 91-186:** CUCUMBER_WITH_SELENIUM_JAVA_STEPS - Generates both Feature file + Step Definition class with WebDriver setup

Playwright TypeScript Implementations:

- **Lines 191-250:** PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY - Generates only Page Object/Fixture classes
- **Lines 256-340+:** PLAYWRIGHT_TYPESCRIPT_FEATURE_PAGE - Generates both Feature file + Test file with Page Object pattern

---

File 2: chat.js

Purpose: Main logic that routes to the correct prompt

Key Methods:

- **Lines 560-605:** getPromptKeys() - Determines which prompt to use based on:
  - Language selection (Java, TypeScript, C#, Python)
  - Engine selection (Selenium, Playwright, Cypress, Puppeteer)

- Generation mode (Feature checkbox / Page checkbox)

c Flow:

```
(Language=TS && Engine=Playwright):
├─ Both checked → PLAYWRIGHT_TYPESCRIPT_FEATURE_PAGE
├─ Feature only → PLAYWRIGHT_TYPESCRIPT_FEATURE_PAGE
└─ Page only → PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY

(Language=Java && Engine=Selenium):
├─ Both checked → CUCUMBER_WITH_SELENIUM_JAVA_STEPS
├─ Feature only → CUCUMBER_ONLY
└─ Page only → SELENIUM_JAVA_PAGE_ONLY
```

ines 607-620: Helper methods:

- `isJavaSelenium()` - Checks Java + Selenium combo
- `isTypescriptPlaywright()` - Checks TypeScript + Playwright combo

3: `panel.html`

ose: UI controls for generation options

JI Elements:

ines 40-50: Generation Mode Checkboxes:

- "Feature File" checkbox
- "Page Class" checkbox

ines 121-130: Language Binding dropdown (Java, C#, Python, TypeScript)

- **Lines 40-50:** Generation Mode Checkboxes:
  - "Feature File" checkbox
  - "Page Class" checkbox
- **Lines 121-130:** Language Binding dropdown (Java, C#, Python, TypeScript)
- **Lines 134-139:** Browser Engine dropdown (Selenium, Playwright, Cypress, Puppeteer)
- **Lines 143-158:** Provider & Model selection (Groq, OpenAI, Claude, Testleaf)
- **Lines 163-177:** API Key input fields

STEP 3: SUMMARY TABLE

Feature	Java + Selenium	TypeScript + Playwright
Page Object Only	✅ SELENIUM_JAVA_PAGE_ONLY	✅ PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY
Feature Only	✅ CUCUMBER_ONLY	✅ PLAYWRIGHT_TYPESCRIPT_FEATURE_PAGE
Feature + Page	✅ CUCUMBER_WITH_SELENIUM_JAVA_STEPS	✅ PLAYWRIGHT_TYPESCRIPT_FEATURE_PAGE

DO YOU APPROVE THIS DETAILED ANALYSIS?

DO YOU APPROVE THIS DETAILED ANALYSIS?

Once you confirm, I will proceed to **STEP 4: Playwright TypeScript Implementation Improvements** where I'll:

- Enhance the PLAYWRIGHT_TYPESCRIPT_FEATURE_PAGE prompt with better test patterns
- Add fixtures for better code organization
- Include proper assertions and waits
- Add more realistic examples
- Ensure production-ready patterns

🎯 Key Improvements Summary:

Aspect	Before	After
Timeouts	None specified	30s nav, 15s elements, 5s assertions
Error Handling	None	Try-catch with logging
Assertions	Basic	Multiple strategies + error assertions
Test Data	Hardcoded	Separate arrays for reusability
Waits	Implicit	Explicit with retry logic
Logging	None	Console.log with emojis
Scenarios	1 positive	2+ positive + negative cases
Type Safety	Any types	Fully typed Locator & Page

📝 **Files Updated:**

- `prompts.js` - Both Playwright TypeScript prompts enhanced

Would you like me to proceed with any additional enhancements or modifications

5️⃣ PLAYWRIGHT_TYPESCRIPT_FEATURE_PAGE ✅

Enhanced in previous step

- ✅ Fixture-based test pattern
- ✅ Multiple test data arrays
- ✅ setup/teardown hooks
- ✅ Data-driven testing
- ✅ Multiple assertion strategies
- ✅ Comprehensive logging
- ✅ Positive + negative scenarios
- ✅ Realistic South India data

📈 Improvement Comparison Table:

Feature	Selenium Java (Before)	Selenium Java (After)	Playwright TS (Before)	Playwright TS (After)
Timeouts	None	30s/15s/500ms configured	Basic	30s/15s/5s configured
Error Handling	None	Try-catch with logging	None	Try-catch with logging
Waits	None	Explicit with polling	Basic	Advanced retry logic